Week 4 at a glance

We will be learning and practicing to:

- Translate between different representations to illustrate a concept.
  - M/W/F – Translating between symbolic and English versions of statements using precise mathematical language
  - Mon – Translating between truth tables (tables of values) and compound propositions
- Use precise notation to encode meaning and present arguments concisely and clearly
  - Mon – Listing the truth tables of atomic boolean functions (and, or, xor, not, if, iff)
  - Wed – Defining functions, predicates, and binary relations using multiple representations
- Know, select and apply appropriate computing knowledge and problem-solving techniques. Reason about computation and systems. Use mathematical techniques to solve problems. Determine appropriate conceptual tools to apply to new situations. Know when tools do not apply and try different approaches. Critically analyze and evaluate candidate solutions.
  - Mon – Evaluating compound propositions
  - Wed – Judging logical equivalence of compound propositions using symbolic manipulation with known equivalences, including DeMorgan's Law
  - Wed – Judging whether a collection of propositions is consistent
  - Mon – Writing the converse, contrapositive, and inverse of a given conditional statement
  - Wed/Fri – Determining what evidence is required to establish that a quantified statement is true or false
  - Wed/Fri – Evaluating quantified statements about finite and infinite domains

TODO:

Review quiz based on Week 3 class material (due Monday)

Homework 2 due Tuesday * Check solutions before Q+A *

Start reviewing for Test 1: **Test 1 is next week**. The test covers material in Weeks 1 through Week 5, corresponding to RQ1, RQ2, RQ3, and RQ4. To study for the exam, we recommend reviewing class notes (e.g. annotations linked on the class website, supplementary videos from the class website) and working multiple variants of review quiz questions. You could also find that reviewing homework (and its posted sample solutions)and extra examples (in lecture notes and discussion examples) and getting feedback (office hours and Piazza) is helpful. **Have you scheduled your test time?**

- You will be required to show a physical photo identification in order to test. Electronic IDs and photos of IDs are not accepted. Additionally, you will only be allowed to bring your ID and a writing utensil into the testing room. You will have to store away all of your other belongings before entering the room. Please see the Triton Testing Center's website for more details.

CC BY-NC-SA 2.0 Version October 15, 2026 (1)

Week 4 Monday: Logical Equivalence and Conditionals

Input					Output	
		Conjunction	Exclusive or	Disjunction	Conditional	Biconditional
p	q	$p \wedge q$	$p \oplus q$	$p \vee q$	$p \to q$	$p \leftrightarrow q$
T	T	T	F	T	T	T
T	F	F	T	T	F	F
F	T	F	T	T	T	F
F	F	F	F	F	T	T
		"p and q"	"p xor q"	"p or q"	"if p then q"	"p if and only if q"

new today

Other ways to represent $p \to q$ column would be in CNF or DNF

The only way to make the conditional statement $p \to q$ false is to have p True and q False.

The **hypothesis** of $p \to q$ is p The **antecedent** of $p \to q$ is p

The **conclusion** of $p \to q$ is q The **consequent** of $p \to q$ is q

The **converse** of $p \to q$ is $q \to p$ Switch the hypothesis and conclusion

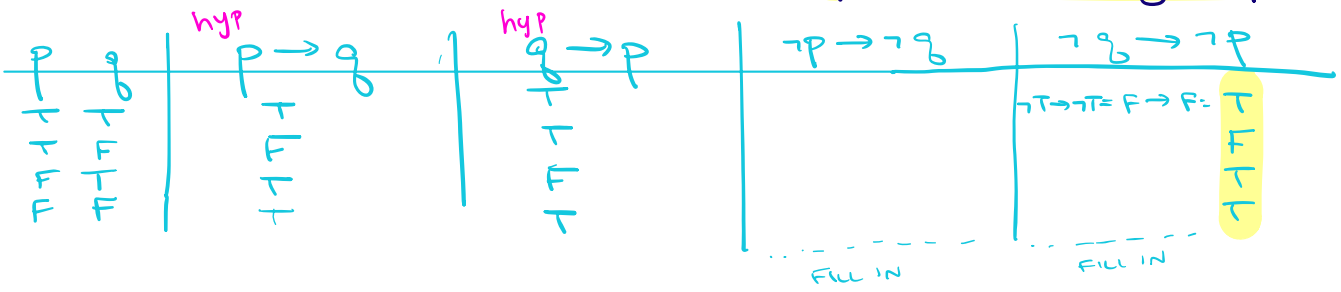
The **inverse** of $p \to q$ is $\neg p \to \neg q$

The **contrapositive** of $p \to q$ is $\neg q \to \neg p$

$p \to q \not\equiv q \to p$

$p \to q \not\equiv \neg p \to \neg q$

$p \to q \equiv \neg q \to \neg p$



We can use a recursive definition to describe all compound propositions that use propositional variables from a specified collection. Here's the definition for all compound propositions whose propositional variables are in $\{p, q\}$.

Basis Step:	p and q are each a compound proposition
Recursive Step:	If x is a compound proposition then so is $(\neg x)$ and if x and y are both compound propositions then so is each of $(x \wedge y), (x \oplus y), (x \vee y), (x \to y), (x \leftrightarrow y)$

Order of operations (Precedence) for logical operators:

Negation, then conjunction / disjunction, then conditional / biconditionals.

Example: $\neg p \vee \neg q$ means $(\neg p) \vee (\neg q)$.

CC BY-NC-SA 2.0 Version October 15, 2026 (2)

(Some) logical equivalences

$p \to q \equiv \neg p \vee q$ ⊛ CNF!

$p \to q \equiv \neg q \to \neg p$ **Contrapositive** — use truth table evaluation, compare columns

LHS RHS

(LHS) $p \to q \equiv \neg p \vee q$ ⊛

$\equiv q \vee \neg p$ commutativity

$\equiv \neg\neg q \vee \neg p$ double negation

$\equiv \neg q \to \neg p$ ⊛

$\equiv \neg q \to \neg p$ (RHS)

$\neg(p \to q) \equiv p \wedge \neg q$

"When is the conditional false?" "only in 2nd row"

$\neg(p \leftrightarrow q) \equiv p \oplus q$

$p \leftrightarrow q \equiv q \leftrightarrow p$

$p \leftrightarrow q \equiv (p \wedge q) \vee (\neg p \wedge \neg q)$

$p \oplus q \equiv (p \wedge \neg q) \vee (\neg p \wedge q)$

p	q	$p \oplus q$	$p \leftrightarrow q$	$\neg(p \leftrightarrow q)$
T	T	F	T	F
T	F	T	F	T
F	T	T	F	T
F	F	F	T	F

De Morgan's: $\neg(p \vee q) \equiv \neg p \wedge \neg q$

$\neg(p \wedge q) \equiv \neg p \vee \neg q$.

Extra examples:

$p \leftrightarrow q$ is not logically equivalent to $p \wedge q$ because when p=F and q=F, $p \leftrightarrow q$ is T but $p \wedge q$ is F.

$p \to q$ is not logically equivalent to $q \to p$ because when p=T and q=F, $p \to q$ is F but $q \to p$ is T.

Goal: work with compound propositions using definitions, metaphors, known logical equivalences

CC BY-NC-SA 2.0 Version October 15, 2026 (3)

Common ways to express logical operators in English:

Negation $\neg p$ can be said in English as

- Not p.
- It's not the case that p.
- p is false.

Conjunction $p \wedge q$ can be said in English as

- p and q.
- Both p and q are true.
- p but q.

Exclusive or $p \oplus q$ can be said in English as

- p or q, but not both.
- Exactly one of p and q is true.

Disjunction $p \vee q$ can be said in English as

- p or q, or both.
- p or q (inclusive).
- At least one of p and q is true.

Conditional $p \rightarrow q$ can be said in English as

- if p, then q.
- p is sufficient for q.
- q when p.
- q whenever p.
- p implies q.
- q follows from p.
- q is necessary for p.
- p only if q.

Biconditional

- p if and only if q.
- p iff q.
- If p then q, and conversely.
- p is necessary and sufficient for q.

CC BY-NC-SA 2.0

Week 4 Wednesday: Translation, Predicates, and Quantifiers

Translation: Express each of the following sentences as compound propositions, using the given propositions.

"A sufficient condition for the warranty to be good is that you bought the computer less than a year ago"

w is "the warranty is good"
b is "you bought the computer less than a year ago"

A sufficient condition for w is b.

$w \not\rightarrow b$

$b \rightarrow w$

"Whenever the message was sent from an unknown system, it is scanned for viruses."

s is "The message is scanned for viruses"
u is "The message was sent from an unknown system"

$u \rightarrow s$

"I will complete my to-do list only if I put a reminder in my calendar"

d is "I will complete my to-do list"
c is "I put a reminder in my calendar"

"If I don't put a reminder in my calendar then I won't complete my to-do list"

$d \rightarrow c$

$c \not\rightarrow d$

Logically equivalent to: $\neg c \rightarrow \neg d$

Not biconditional because don't have "if and only if"

CC BY-NC-SA 2.0 Version October 15, 2026 (5)

at least one

Definition: A collection of compound propositions is called **consistent** if there is an assignment of truth values to the propositional variables that makes each of the compound propositions true.

Consistency:

collection of statements

Whenever the system software is being upgraded, users cannot access the file system. If users can access the file system, then they can save new files. If users cannot save new files, then the system software is not being upgraded.

1. Translate to symbolic compound propositions

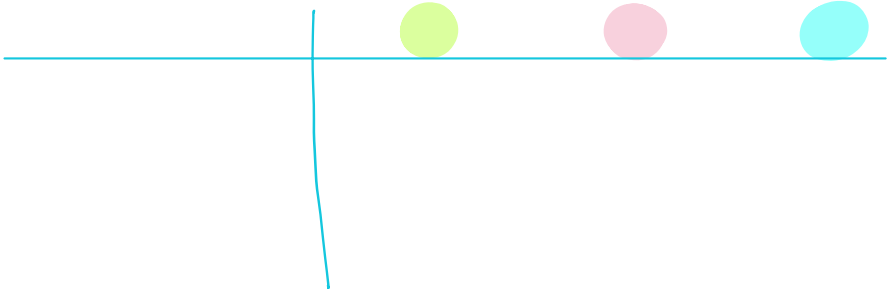
u = system software is being upgraded

a = users can access file system

n = users can save new files

$u \rightarrow \neg a$ $\quad a \rightarrow n$ $\quad \neg n \rightarrow \neg u$

2. Look for some truth assignment to the propositional variables for which all the compound propositions output T



OR

look for a row where all outputs T

eg. for conditionals we can work to make conclusion T

There is a truth assignment to a, n, u for which all the compound propositions output T

So the collection of compound propositions is consistent.

a = F

n = T

u = F

CC BY-NC-SA 2.0

Definition: A **predicate** is a function from a given set (domain) to $\{T, F\}$.

A predicate can be applied, or **evaluated** at, an element of the domain.

Usually, a predicate *describes a property* that domain elements may or may not have.

Two predicates over the same domain are **equivalent** means they evaluate to the same truth values for all possible assignments of domain elements to the input. In other words, they are equivalent means that they are equal as functions.

To define a predicate, we must specify its domain and its value at each domain element. The rule assigning truth values to domain elements can be specified using a formula, English description, in a table (if the domain is finite), or recursively (if the domain is recursively defined).

Note: Since logical operations are functions, they can be part of the definitions of predicates.

3 predicates

Input	Output		
	$V(x)$	$N(x)$	$Mystery(x)$
x	$[x]_{2c,3} > 0$	$[x]_{2c,3} < 0$	
000	F	F	T
001	T	F	T
010	T	F	T
011	T	F	F
100	F	T	F
101	F	T	T
110	F	T	F
111	F	T	T

8 rows

The domain for each of the predicates $V(x)$, $N(x)$, $Mystery(x)$ is $\{000, 001, 010, 011, 100, 101, 110, 111\}$.

Fill in the table of values for the predicate $N(x)$ based on the formula given.

When is x the 2s complement width 3 representation of a negative number?

Definition: The **truth set** of a predicate is the collection of all elements in its domain where the predicate evaluates to T.

Notice that specifying the domain and the truth set is sufficient for defining a predicate.

The truth set for the predicate $V(x)$ is $\{001, 010, 011\}$.

The truth set for the predicate $N(x)$ is $\{100, 101, 110, 111\}$.

The truth set for the predicate $Mystery(x)$ is $\{000, 001, 010, 101, 111\}$.

CC BY-NC-SA 2.0

"for all"

The **universal quantification** of predicate $P(x)$ over domain U is the statement "$P(x)$ for all values of x in the domain U" and is written $\forall x P(x)$ or $\forall x \in U\ P(x)$. When the domain is finite, universal quantification over the domain is equivalent to iterated *conjunction* (ands).

The **existential quantification** of predicate $P(x)$ over domain U is the statement "There exists an element x in the domain U such that $P(x)$" and is written $\exists x P(x)$ for $\exists x \in U\ P(x)$. When the domain is finite, existential quantification over the domain is equivalent to iterated *disjunction* (ors).

"there exists"

Statements involving predicates and quantifiers are **logically equivalent** means they have the same truth value no matter which predicates (domains and functions) are substituted in.

Quantifier version of De Morgan's laws: $\boxed{\neg\forall x P(x) \equiv \exists x\,(\neg P(x))}$ $\boxed{\neg\exists x Q(x) \equiv \forall x\,(\neg Q(x))}$

An element for which $P(x) = F$ is called a **counterexample** of $\forall x P(x)$. When there is a counterexample in the domain of predicate P, $\forall x P(x)$ is false.

An element for which $P(x) = T$ is called a **witness** of $\exists x P(x)$. When there is a witness in the domain of predicate P, then $\exists x P(x)$ is true.

Examples of quantifications using $V(x), N(x), Mystery(x)$: Examples on Friday...

True or **False**: $\exists x\ (\ V(x) \vee N(x)\ )$ witness: 011 b/c V(011) ∨ N(011) is true.

True or **False**: $\forall x\ (\ V(x) \vee N(x)\ )$ counterexample: 000 b/c V(000) ∨ N(000) is false

True or **False**: $\exists x\ (\ \neg V(x) \wedge \neg N(x)\ )$ witness: 000 b/c ¬V(000) ∧ ¬N(000) is true.

$\equiv \neg\forall x\ (V(x) \vee N(x))$

True or **False**: $\forall x\ (\ V(x) \rightarrow N(x)\ )$ counterexample: 001 b/c V(001) is True and N(001) is false

True or **False**: $\exists x\ (\ N(x) \leftrightarrow Mystery(x)\ )$ witness: 011 b/c N(011) is false and Mystery(011) is false

Rewrite $\neg\forall x\ (\ V(x) \oplus Mystery(x)\ )$ into a logically equivalent statement.

[

Notice that these are examples where the predicates have *finite* domain. How would we evaluate quantifications where the domain may be infinite?

CC BY-NC-SA 2.0

Week 4 Friday: Evaluating Nested Quantifiers

Recall the definitions: The set of RNA strands S is defined (recursively) by:

Basis Step:	$\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$
Recursive Step:	If $s \in S$ and $b \in B$, then $sb \in S$

where sb is string concatenation.

$B = \{A, C, U, G\}$

The function *rnalen* that computes the length of RNA strands in S is defined recursively by:

		$rnalen : S$	$\to \mathbb{Z}^+$
Basis Step:	If $b \in B$ then	$rnalen(b)$	$= 1$
Recursive Step:	If $s \in S$ and $b \in B$, then	$rnalen(sb)$	$= 1 + rnalen(s)$

The function *basecount* that computes the number of a given base b appearing in a RNA strand s is defined recursively by:

		$basecount : S \times B$	$\to \mathbb{N}$
Basis Step:	If $b_1 \in B, b_2 \in B$	$basecount(\ (b_1, b_2)\ )$	$= \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$
Recursive Step:	If $s \in S, b_1 \in B, b_2 \in B$	$basecount(\ (sb_1, b_2)\ )$	$= \begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$

Example predicates on S, the set of RNA strands (an infinite set)

$L_{\mathtt{A}} : S \to \{T, F\}$ defined recursively by:

Basis step: $L_{\mathtt{A}}(\mathtt{A}) = T$, $L_{\mathtt{A}}(\mathtt{C}) = L_{\mathtt{A}}(\mathtt{G}) = L_{\mathtt{A}}(\mathtt{U}) = F$

Recursive step: If $s \in S$ and $b \in B$, then $L_{\mathtt{A}}(sb) = L_{\mathtt{A}}(s)$.

Example where $L_{\mathtt{A}}$ evaluates to T is A or AGA

$L_A(AGA) = L_A(AG) = L_A(A) = T$

Example where $L_{\mathtt{A}}$ evaluates to F is CGG

$L_A(CGG) = L_A(CG) = L_A(C) = F$

$H : S \to \{T, F\}$ where $H(s) = T$ for all s.

Truth set of H is S

A
C
U
G
AA
AC
AU
AG
CA
⋮

CC BY-NC-SA 2.0

Notation: for a predicate P with domain $X_1 \times \cdots \times X_n$ and a n-tuple $(x_1, \ldots, x_n)$ with each $x_i \in X$, we can write $P(x_1, \ldots, x_n)$ to mean $P(\ (x_1, \ldots, x_n)\ )$.

L with domain $S \times \mathbb{Z}^+$ is defined by, for $s \in S$ and $n \in \mathbb{Z}^+$,

$$L(\ (s, n)\ ) = \begin{cases} T & \text{if } rnalen(s) = n \\ F & \text{otherwise} \end{cases}$$

In other words, $L(\ (s, n)\ )$ means $rnalen(s) = n$

BC with domain $S \times B \times \mathbb{N}$ is defined by, for $s \in S$ and $b \in B$ and $n \in \mathbb{N}$,

$$BC(\ (s, b, n)\ ) = \begin{cases} T & \text{if } basecount(\ (s, b)\ ) = n \\ F & \text{otherwise} \end{cases}$$

In other words, $BC(\ (s, b, n)\ )$ means $basecount(\ (s, b)\ ) = n$

Example where L evaluates to T: (A, 1) Why? A∈S, 1∈ℤ⁺, rnalen(A)=1

Example where BC evaluates to T: (ACA, A, 2) Why? ACA∈S, A∈B, basecount((ACA,A)) = 2.

Example where L evaluates to F: (A, 2) Why? A∈S, 2∈ℤ⁺, rnalen(A)≠2.

Example where BC evaluates to F: (ACA, A, 1) Why? ACA∈S, A∈B, basecount((ACA,A)) ≠ 1.

$\exists t\ BC(t)$ $\qquad$ $\exists (s, b, n) \in S \times B \times \mathbb{N}\ (basecount(\ (s, b)\ ) = n)$

In English: There is some choice of strand s, base b, nonnegative integer n where the equations basecount((s,b)) = n.

Witness that proves this existential quantification is true: (ACA, A, 2)

$\forall t\ BC(t)$ $\qquad$ $\forall (s, b, n) \in S \times B \times \mathbb{N}\ (basecount(\ (s, b)\ ) = n)$

In English: For all choices of strand s, base b, nonnegative int n the equation basecount((s,b)) = n.

Counterexample that proves this universal quantification is false: (ACA, A, 1)

CC BY-NC-SA 2.0

$S \times B \times \mathbb{N}$

New predicates from old: Consider several different predicates that are related to BC:

1. Define the **new** predicate with domain $S \times B$ and rule

   $$basecount(\ (s, b)\ ) = 3$$

   Example domain element where predicate is T: (AAA, A)

2. Define the **new** predicate with domain $S \times \mathbb{N}$ and rule

   $$basecount(\ (s, \mathtt{A})\ ) = n$$

   Example domain element where predicate is T: (A, 1)

   $\exists n \in \mathbb{N}$ (basecount ((AA, A)) = n)
   $\exists n \in \mathbb{N}$ (basecount ((AA, C)) = n)

3. Define the **new** predicate with domain $S \times B$ and rule

   $$\exists n \in \mathbb{N}\ (basecount(\ (s, b)\ ) = n)$$

   Example domain element where predicate is T: (AA, A) or (AA, C)

   notice: any choice of (strand, base) works!

4. Define the **new** predicate with domain S and rule

   $$\forall b \in B\ (basecount(\ (s, b)\ ) = 1)$$

   Example domain element where predicate is T: ACGU or CGAU

Nested quantifiers

$$\forall s \in S\ \forall b \in B\ \forall n \in \mathbb{N}\ (basecount(\ (s, b)\ ) = n)$$

In English: For all strands s, bases b, and numbers n, basecount((s,b)) = n

Counterexample that proves this universal quantification is false:

s = AC b = A n = 2

$$\forall n \in \mathbb{N}\ \forall s \in S\ \forall b \in B\ (basecount(\ (s, b)\ ) = n)$$

In English: For all numbers n, strands s, bases b, basecount((s,b)) = n

Counterexample that proves this universal quantification is false:

s = AC b = A n = 2

CC BY-NC-SA 2.0

FOR MONDAY....

Alternating nested quantifiers

$$\forall s \in S \; \exists b \in B \; (\; basecount(\; (s, b) \;) = 3 \;)$$

In English: For each RNA strand there is a base that occurs 3 times in this strand.

Write the negation and use De Morgan's law to find a logically equivalent version where the negation is applied only to the BC predicate (not next to a quantifier).

Is the original statement **True** or **False**?

$$\exists s \in S \; \forall b \in B \; \exists n \in \mathbb{N} \; (\; basecount(\; (s, b) \;) = n \;)$$

In English: There is an RNA strand so that for each base there is some nonnegative integer that counts the number of occurrences of that base in this strand.

Write the negation and use De Morgan's law to find a logically equivalent version where the negation is applied only to the BC predicate (not next to a quantifier).

Is the original statement **True** or **False**?

CC BY-NC-SA 2.0